

Modern Development Environment Setup

GitHub, VS Code, Claude AI, Python & R

Lumumba W. Victor
Trainer — Data Analyst — ML Expert

MediaCrest Training College

July 22, 2026

Abstract

This session provides a comprehensive guide to setting up a modern, AI-assisted development environment. We transition from basic version control to building an integrated workspace that leverages cloud collaboration and artificial intelligence.

- **GitHub:** Establishing robust version control and collaboration workflows.
- **VS Code:** Configuring a lightweight, extensible, and powerful central IDE.
- **Claude AI:** Integrating state-of-the-art AI for code generation and debugging.
- **Execution:** Seamlessly running and managing Python and R scripts natively.

Our ultimate goal is to create a highly productive, leak-proof, and intelligent coding ecosystem for data science and software development.

Session Objectives

- Create, configure, and navigate a GitHub account and repository structure.
- Download, install, and customize Visual Studio Code for optimal productivity.
- Seamlessly connect VS Code to GitHub for version control and team collaboration.
- Integrate Claude AI into the VS Code workflow to accelerate coding and debugging.
- Set up, configure, and execute Python and R scripts within the integrated development environment.
- Understand best practices for multi-language development and environment management.

Agenda

- ➊ **GitHub Setup & Basics** (Account creation, repositories, version control)
- ➋ **VS Code Installation & Configuration** (Setup, themes, essential extensions)
- ➌ **Connecting VS Code to GitHub** (Authentication, cloning, committing, PRs)
- ➍ **Integrating Claude AI in VS Code** (Extensions, API setup, AI-assisted coding)
- ➎ **Running Python and R Code** (Interpreter setup, execution, Jupyter integration)

Part 1: GitHub Setup & Basics

"Establishing the foundation for version control and collaboration."

What is GitHub?

- **Cloud-Based Platform:** A web-based hosting service for version control using Git.
- **Version Control:** Tracks every modification to the code in a special database.
- **Collaboration Hub:** Allows multiple developers to work on the same project simultaneously.
- **Industry Standard:** Essential for modern software development, data science, and open-source research.

Core Concept

If Git is the engine that tracks changes locally, GitHub is the cloud garage where your project is stored, shared, and collaboratively built.

Why Use GitHub?

- **Version Control:** Revert to previous states, track bugs, and manage releases safely.
- **Collaboration:** Work concurrently without overwriting each other's code.
- **Portfolio Building:** Showcase your projects, contributions, and coding style to employers.
- **Open Source:** Contribute to global projects, leverage community tools, and build networks.

The Collaboration Imperative

In modern data science and engineering, isolated coding is obsolete. GitHub enables peer review, continuous integration, and seamless team scaling.

Signing Up for GitHub

- ➊ **Navigate:** Visit `github.com` and click the "Sign up" button.
- ➋ **Credentials:** Enter a valid email, create a secure password, and choose a unique username.
- ➌ **Verification:** Complete the user verification puzzle to prove you are human.
- ➍ **Email Confirmation:** Check your inbox and verify your email address to activate the account.
- ➎ **Plan Selection:** Choose the Free tier (sufficient for public/private repos and unlimited collaborators).

Profile & Repository Basics

- **Profile:** Add a professional bio, profile picture, and links to your portfolio or LinkedIn.
- **Repositories (Repos):** The fundamental folders where project files and revision history live.
- **Visibility:**
 - *Public:* Visible to everyone; ideal for open-source and portfolios.
 - *Private:* Visible only to you and invited collaborators; for proprietary work.
- **README.md:** The landing page of your repository. Crucial for documenting project goals and setup instructions.

Part 2: VS Code Installation & Configuration

"Setting up the central hub for your development workflow."

What is VS Code?

- **Lightweight yet Powerful:** A free, open-source source code editor by Microsoft.
- **Cross-Platform:** Natively available on Windows, macOS, and Linux.
- **Extensible:** Features a massive marketplace with thousands of extensions for every language.
- **Integrated Tooling:** Built-in support for Git, debugging, terminal, and intelligent code completion (IntelliSense).

The Developer's Swiss Army Knife

VS Code strikes the perfect balance between a simple text editor and a heavy, resource-intensive Integrated Development Environment (IDE).

Downloading VS Code

- **Official Source:** Visit `code.visualstudio.com`.
- **Auto-Detection:** The site automatically detects your operating system and architecture.
- **Stable Build:** Download the "Stable" version (avoid "Insiders" unless you want beta features).
- **Installer Type:**
 - *Windows:* User Installer is recommended (doesn't require admin rights).
 - *macOS/Linux:* Download the standard `.zip` or `.tar.gz` archive.

Installing VS Code

- **Windows:** Run the .exe. *Crucial:* Check "Add to PATH" and "Register Code as editor for supported file types".
- **macOS:** Drag the app to the Applications folder. Use the Command Palette to run "Shell Command: Install 'code' in PATH".
- **Linux:** Install via .deb/.rpm packages, or use Snap/Flatpak for easy updates.

Post-Installation

Launch the application. The first run will guide you through basic telemetry settings and theme selection.

Initial Setup & Extensions

- **Theme & Font:** Customize for readability (e.g., One Dark Pro theme, Fira Code font with ligatures).
- **Essential Extensions:**
 - **Python:** Official Microsoft extension for Python support.
 - **R:** R Debugger / R Extension for syntax and execution.
 - **GitLens:** Supercharges the built-in Git capabilities.
 - **Prettier:** Automated code formatting for consistent style.

Extension Marketplace

Access via the sidebar icon (Ctrl+Shift+X). Search, install, and reload if prompted.

Part 3: Connecting VS Code to GitHub

"Connecting your local editor to the cloud for seamless version control."

GitHub Integration in VS Code

- **Built-in Git:** VS Code has native Git support for basic staging, committing, and pushing.
- **GitHub Pull Requests & Issues:** An official extension to manage PRs, review code, and handle issues without leaving the editor.
- **GitHub Repositories:** Allows you to browse and edit remote repositories directly without cloning them locally.

The Workflow

We will primarily use the built-in Git features combined with GitHub authentication for a streamlined, native experience.

Authenticating with GitHub

- ❶ Open the Command Palette (Ctrl+Shift+P or Cmd+Shift+P).
- ❷ Type and select GitHub: Sign in to GitHub.
- ❸ A browser window will open prompting you to authorize VS Code.
- ❹ Click "Authorize" and return to VS Code.
- ❺ Alternatively, use a Personal Access Token (PAT) via the Git Credential Manager if browser auth fails.

Cloning a Repository

- **Source Control Panel:** Click the "Clone Repository" icon on the left sidebar.
- **Command Palette:** Run `Git: Clone` and paste the GitHub URL (HTTPS or SSH).
- **Local Directory:** Choose a folder on your machine where the project will be downloaded.
- **Open:** VS Code will prompt you to open the cloned folder. Always click "Open" to set the workspace root.

SSH vs HTTPS

SSH is preferred for frequent pushes as it uses key-based authentication, avoiding repeated password prompts.

Committing and Pushing Changes

- **Stage:** Click the + next to modified files in the Source Control tab to stage them.
- **Message:** Type a clear, descriptive commit message in the text box.
- **Commit:** Click the checkmark (✓) to save the changes to your local Git history.
- **Push:** Click "Sync Changes" or "Push" to upload your local commits to the GitHub remote.
- **Pull:** Always pull remote changes before starting new work to avoid merge conflicts.

Branching and Pull Requests

- **Branches:** Isolate features (e.g., feature/data-cleaning). Create via the bottom-left status bar.
- **Push Branch:** Push your local branch to GitHub to make it visible to the team.
- **Pull Request (PR):** Created on GitHub.com to propose merging your branch into main.
- **Review:** Use the VS Code PR extension to review diffs, leave comments, and approve code.

Best Practice

Never commit directly to `main`. Always use feature branches and Pull Requests for code review.

Part 4: Integrating Claude AI in VS Code

"Supercharging your coding with AI assistance."

What is Claude AI?

- **Developer:** Built by Anthropic, designed to be helpful, harmless, and honest.
- **Capabilities:** Excels in complex reasoning, coding, mathematics, and natural language understanding.
- **Models:** State-of-the-art models like Claude 3.5 Sonnet and Claude 3 Opus.
- **Context Window:** Massive context limits allow it to understand entire codebases at once.

The AI Advantage

Claude acts as an expert pair-programmer, available 24/7 to explain, generate, and refine your code.

Why Use AI in Coding?

- **Productivity:** Automate boilerplate code, write tests, and generate documentation instantly.
- **Learning:** Understand complex codebases, new libraries, and obscure syntax quickly.
- **Debugging:** Identify edge cases, trace root causes of errors, and suggest robust fixes.
- **Refactoring:** Improve code readability, performance, and adherence to best practices.

The Paradigm Shift

AI doesn't replace the developer; it elevates them. The skill shifts from writing syntax to architecting solutions and reviewing AI output.

Installing an AI Extension

- VS Code requires an extension to connect to Claude's API.
- **Continue:** A leading open-source AI code assistant. Highly customizable, supports Claude via API.
- **Cline:** An autonomous coding agent that can execute terminal commands and edit files, also supporting Claude.
- **Installation:** Search for "Continue" or "Cline" in the VS Code Extensions marketplace and click Install.

Choosing the Right Tool

Use *Continue* for inline chat and completions. Use *Cline* if you want an autonomous agent to build features end-to-end.

Configuring Claude API Keys

- ➊ **Get API Key:** Visit `console.anthropic.com`, create an account, and generate an API key.
- ➋ **Open Settings:** In VS Code, open the extension settings (e.g., Continue's `config.json`).
- ➌ **Add Provider:** Add Anthropic as a provider and paste your API key securely.
- ➍ **Select Model:** Choose your preferred model (e.g., `claude-3-5-sonnet-20241022`).
- ➎ **Save & Reload:** Save the configuration and reload VS Code to activate the connection.

Using Claude for Code Generation

- **Inline Generation:** Highlight code or press `Ctrl+I` (`Cmd+I`) to generate or edit code in-place.
- **Chat Panel:** Open the sidebar chat to ask architectural questions or design algorithms.
- **Context Injection:** Use `@files` or `@codebase` to give Claude specific context about your project.
- **Prompt Engineering:** Be specific. "Write a Python function to parse this JSON and handle missing keys."

Debugging and Refactoring

- **Debugging:** Paste error logs or stack traces into the chat. Claude can trace the root cause and suggest fixes.
- **Refactoring:** Highlight a block and ask, "Refactor this to be more Pythonic and PEP 8 compliant."
- **Documentation:** "Add comprehensive docstrings and type hints to this function."
- **Test Generation:** "Write unit tests for this R function using the `testthat` framework."

Review is Crucial

Always review AI-generated code. AI can hallucinate libraries or introduce subtle logical bugs.

Part 5: Executing Python and R Code

"Running data science and machine learning workflows natively."

Setting up Python in VS Code

- **Install Python:** Download from `python.org`. *Crucial:* Check "Add Python to PATH".
- **Install Extension:** Ensure the official "Python" extension by Microsoft is installed.
- **Select Interpreter:** Open Command Palette → Python: Select Interpreter.
- **Virtual Environments:** Always select a virtual environment (`venv` or `conda`) to isolate project dependencies.

Environment Isolation

Never install project packages in the global Python environment. It leads to version conflicts and broken projects.

Running Python Code

- **Create File:** Create a new file with a `.py` extension.
- **Write Code:** Write your script (e.g., `print("Hello, Data Science!")`).
- **Run Script:** Click the "Run" (Play) icon in the top-right corner, or press `Ctrl+F5`.
- **Interactive Window:** Use

Debugging

Set breakpoints by clicking the left margin. Press `F5` to start the debugger and step through your code.

Setting up R in VS Code

- **Install R:** Download and install R from CRAN (cran.r-project.org).
- **Install Extensions:** Install the "R" extension (by Yuki Ueda) and "R Debugger".
- **Radian Console:** Install radian via pip (`pip install radian`) for a modern, syntax-highlighted R console.
- **Configure Path:** If R is not auto-detected, manually set the `r.rpath.mac` / `r.rpath.windows` in VS Code settings.

Running R Code

- **Create File:** Create a new file with an .R extension.
- **Write Code:** Write your script (e.g., `summary(mtcars)`).
- **Run Line/Selection:** Place the cursor on a line or highlight code, then press `Ctrl+Enter` (`Cmd+Enter`).
- **Run Source:** Use the "Run Source" button in the editor toolbar to execute the entire script.
- **Terminal:** Use the integrated terminal to run interactive R sessions or Radian.

Jupyter Notebooks in VS Code

- **Native Support:** VS Code natively supports `.ipynb` files without needing JupyterLab.
- **Jupyter Extension:** Install the official "Jupyter" extension for full functionality.
- **Multi-Language:** Mix Python and R (via the `IRkernel`) in the same notebook interface.
- **Rich Outputs:** View plots, interactive dataframes, and rendered Markdown directly in the editor.

Exploratory Data Analysis

Notebooks in VS Code are ideal for EDA, prototyping machine learning models, and sharing analytical results.

Best Practices for Multi-Language Dev

- **Workspaces:** Use VS Code Multi-Root Workspaces to manage complex, multi-language projects.
- **Environments:** Always use virtual environments (Python `venv/conda`) and `renv` (R).
- **Linting & Formatting:** Enable `flake8/black` (Python) and `lintr` (R) for clean code.
- **Gitignore:** Ensure environments, cache folders, and API secrets are excluded from GitHub commits.
- **AI Verification:** Treat AI-generated code as a draft. Always test and verify before merging.

Questions?

Lumumba W. Victor

Trainer — Data Analyst — ML Expert

MediaCrest Training College